

WEB WORKERS

Keep the Main Thread Free with Web Workers



THE PLAN

Today's path

Feel the freeze.
Prove it.
Move the work.

1 Feel the freeze

Make the stutter obvious

3 Move the work

Main thread $\Rightarrow$ worker

5 Use the pattern

Interactive app proof

2 Prove the blocker

DevTools receipt

4 Add guardrails

Progress · cancel · debounce



<https://github.com/cyatteau/kcdc2026-web-workers>

Jank is a workload problem.

Users feel it before we diagnose it.

Delayed clicks

Frozen motion

Laggy interactions

DevTools sees: **one big JS task**

WHY IT FREEZES

One task.

Everything behind it waits.

Input

Paint

Motion

Long task
runs to completion

THREAD

A practical rescue path.



LIVE DEMO 01

THREAD: T

DEMO

01

Trigger the stutter

Just the browser under pressure.

WATCH FOR

Heartbeat Freezes

Click waits and typing box

The page gave us a receipt.

Every symptom points to the same
blocked thread.

Heartbeat paused

Click waited

Typing queued

Next: check DevTools

One front desk.

The main thread handles the experience.

Clicks

Paint

UI
updates

Heavy JS
blocks the line

Async does not move CPU work

Waiting and calculating are different jobs.

Promises

async / await can pause
CPU work still runs here

Web Worker

Separate execution context
Built for sustained CPU work

Async syntax $\neq$ parallel execution

Prove it in DevTools.

Record the interaction, then follow the longest task.

Record a trace

Reproduce the lag

Find the long task

Inspect the task

BEFORE A WORKER

Three ways to rescue the UI.

Choose the smallest fix that solves the problem.

Do less

Remove work

Break it up

Yield often

Move it

Use a worker

Not a second UI.

A separate workspace for data chores.

Main thread

DOM + UI
clicks + paint

request →

← result

Worker thread

plain data
calculation

Send data, not behavior

Messages cross through structured cloning.

Works well

Objects and arrays
Typed data

Does not cross

Functions
DOM nodes
Live UI objects

`postMessage()` defines the boundary

Move the work

Same workload. Different thread.

MAIN

freezes

WORKER

breathes

Make worker messages boring.

Boring is good when two threads talk.

```
{  
  type: "ANALYZE",  
  requestId: "req-001",  
  payload: {...}  
}
```

START

PROGRESS

DONE/CANCEL

Workers help. Design still matters.

A worker is only one part of the pattern.

Debounce

Wait

Progress

Inform

Cancel

Stop stale

Failure needs a UI path.

A worker can fail without freezing the page.

Report

Send error data

Recover

Show a fallback

Clean up

Reset the state

“Cancel” can mean two things

Stop computation, or stop trusting its answer.

Hard stop

Terminate the worker
Recreate it for the next job

Soft cancel

Keep the worker alive
Ignore stale request IDs

Choose based on cost and restart behavior

Fast filters

A worker filter flow with debounce.

search: parks...

Wait for a typing pause

Old results ignored

STALE RESULTS

Newest request owns the UI.

Completion order is not the same as user intent.

Request A starts

Request B starts

B finishes → render

A finishes: ignore

What are we scoring?

Park boundary records for review priority

Question

Which visible boundary records should be reviewed first?

Size

larger footprint

bigger spatial impact

Complexity

more rings + vertices

harder to inspect

Data confidence

quality metadata

contributes to score

Visible boundaries → score + sort records → highlight top results

Copy or transfer?

Large payloads can change the math.

Clone

Broad data support
Sender keeps its copy

Transfer

Useful for large buffers
Sender loses access

Measure message cost before optimizing it

QUICK CHECK

Would you move it?

Worker?
Main thread?
Measure first?

1 5 ms task

Predictable calculation

3 100k-row sort

Sustained CPU work

5 Pixel transform

Large image buffer

2 fetch() call

Mostly waiting

4 DOM reads

Needs the page

DECIDE

Keep the experience on the main thread

Move expensive data chores when they earn the trip.

Good fit

Plain data in, plain data out
UI needs to stay alive

Probably overkill

Tiny calculations
Normal async calls

THREAD

A practical rescue path.



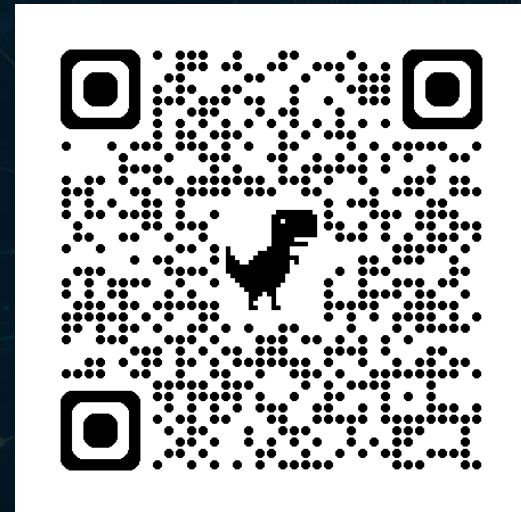
Thank you, KCDC!

Courtney Yatteau

  @c_yatteau

 courtneyyatteau

 @cyatteau.bsky.social



[https://github.com/cyatteau/
kcdc2026-web-workers](https://github.com/cyatteau/kcdc2026-web-workers)